SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4307
STUDENT NAME: K. Ganesh
REGISTER NUMBER: 192411170

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: **Total Marks: 20**

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with `Content-Length` and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

2: **DOM Structure Analysis**

Consider the following DOM tree representation:

```
<html>
  <head>
  <title>TestPage</title>
</head>
  <body>
  <div>
  <p>HelloWorld
  </div>
 </body>
 </html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Code of Conduct:

I K. Ganesh / 192411170 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: *K. Ganesh*

RUBRICS FOR CALCULATION:

Criteria	Weigh tage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Question 1: HTTP Response Analysis

The server returned this header: HTTP/1.1 200 OK, Date: Mon, 01 Apr 2026 10:00:00 GMT, Content-Type: text/html, Content-Length: -150, Connection: keep-alive.

1. Identify any incorrect or invalid fields in the header.

The clearly invalid field is **`Content-Length: -150`**. This header counts the bytes in the message body, so it has to be a whole number that is zero or more. A minus sign is simply not allowed. There is also a smaller slip in the Date field: 1 April 2026 falls on a Wednesday, not a Monday, so the day name and the date disagree. The status line, Content-Type and Connection fields are all written correctly.

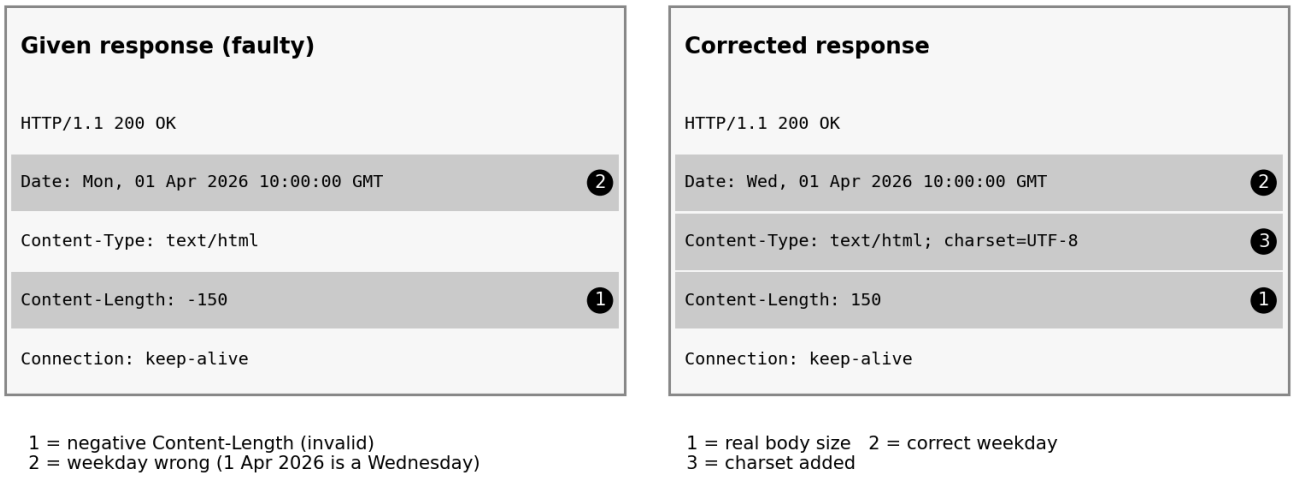


Figure 1: Faulty HTTP response header compared with the corrected one

2. Analyze the issue with Content-Length and explain its impact.

Content-Length tells the client exactly how many bytes to read before the response ends. A negative number has no meaning, so the browser cannot tell where the body stops. This matters more here because of **`Connection: keep-alive`**, which keeps the same TCP connection open for the next request. If the length is wrong, the client may reject the response, show a half-loaded page, wait forever for bytes that never come, or start reading the next response from the wrong position. Proxies usually refuse such a response with a 502 Bad Gateway error, and mismatched lengths are also a known ingredient in request smuggling attacks.

3. Determine whether the status code matches the response correctness.

200 OK is a real status code and on its own it says the request succeeded. But the response as a whole is not correct, because the broken Content-Length contradicts that claim of success. So the status code and the actual quality of the response do not match. If the server really cannot build a valid response, an honest 500 Internal Server Error is better. Once the header is fixed, 200 OK is the right code.

4. Suggest corrections for the identified errors.

- Set Content-Length to the real size of the body in bytes (for example **`Content-Length: 150`**), or leave it out and use **`Transfer-Encoding: chunked`** when the size is not known in advance.
- Correct the date to **`Wed, 01 Apr 2026 10:00:00 GMT`**. It is best to let the server generate this from its own clock.
- Add a character set: **`Content-Type: text/html; charset=UTF-8`**.
- Check the fixed headers with **`curl -I`** or the Network tab of the browser developer tools.

> **Conclusion:** The response goes wrong mainly because of the negative Content-Length, with a mismatched weekday as a minor issue. After the fixes, the headers and the 200 OK status agree with each other and any client can read the body safely.

Question 2: DOM Structure Analysis

1. Identify missing or improperly closed elements.

The **<p>** tag is opened but never closed. Every other element (html, head, title, body and div) has its closing tag. Two smaller things are also worth noting: the text "HelloWorld" looks like two words joined without a space, and the page has no DOCTYPE declaration.

2. Analyze how the browser will interpret this structure.

Browsers do not stop on bad HTML. The HTML5 parser follows fixed error-recovery rules and builds the tree by itself. When it reaches ** ** while a paragraph is still open, it closes the paragraph first and then the div. The final tree is therefore html, then head with title, and body with div, p and the text. It ends up the same as the corrected tree in Figure 2. A strict XML or XHTML parser would behave differently: it would treat the missing tag as a well-formedness error and refuse to display the page.

3. Determine the impact on rendering and layout.

For this tiny page the visible result is almost identical. The paragraph is a block element, so the text appears on its own line inside the div with the default top and bottom margin. The risk grows in bigger pages. If more content follows an unclosed paragraph, the browser decides where it ends, and that can change spacing, nesting and CSS selectors such as **div > p**. Different parsers may also guess differently, which leads to inconsistent layouts. The missing DOCTYPE pushes the browser into quirks mode, which can change box sizing and spacing as well.

4. Suggest corrections to make the DOM valid.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>TestPage</title>
</head>
<body>
  <div>
    <p>Hello World</p>
  </div>
</body>
</html>
```

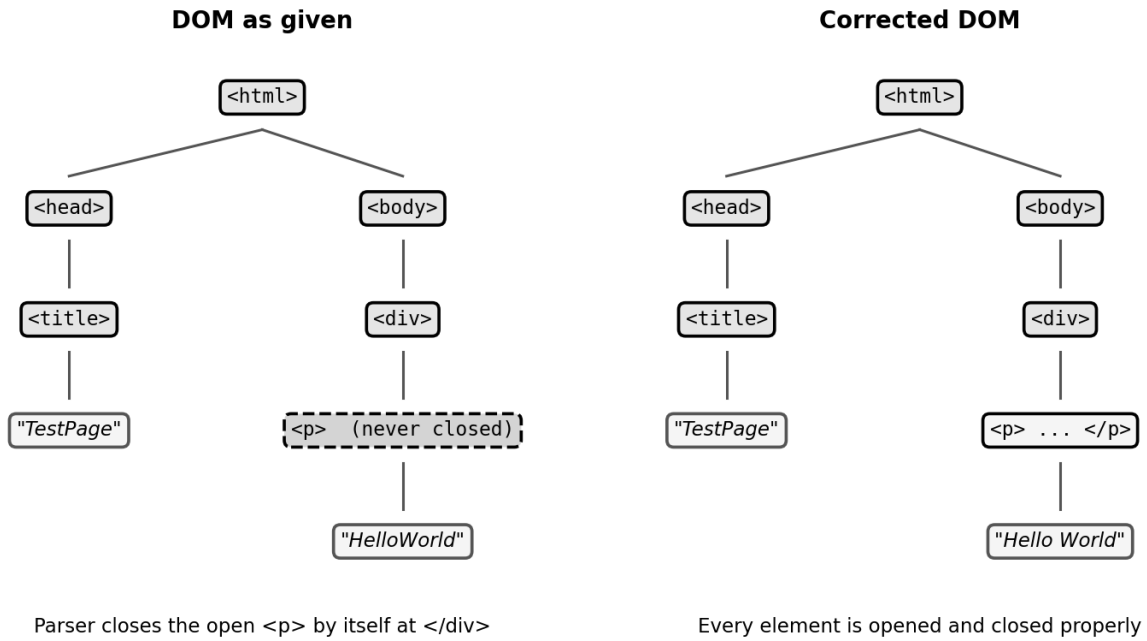


Figure 2: DOM tree as given and the corrected DOM tree

Conclusion: The only real structural error is the unclosed p element. The browser repairs it silently, but a valid document should not depend on error recovery. Closing the tag and adding the DOCTYPE makes the structure predictable in every browser.

Question 3: GET vs POST in Login System

1. Interpret the differences in data transmission between GET and POST.

With GET, the form data is appended to the URL as a query string, so it travels in the request line itself. With POST, the same data is placed inside the request body, after the headers. The URL stays clean, and the Content-Type header describes the body.

Point	GET	POST
Where data goes	In the URL (query string)	In the request body
Visibility	Visible in address bar and history	Not shown in the URL
Stored in	Browser history, bookmarks, server and proxy logs	Normally not stored in URL logs
Size limit	Limited by URL length	No practical limit
Intended use	Fetching or searching data (safe, repeatable)	Sending or changing data (login, forms, uploads)

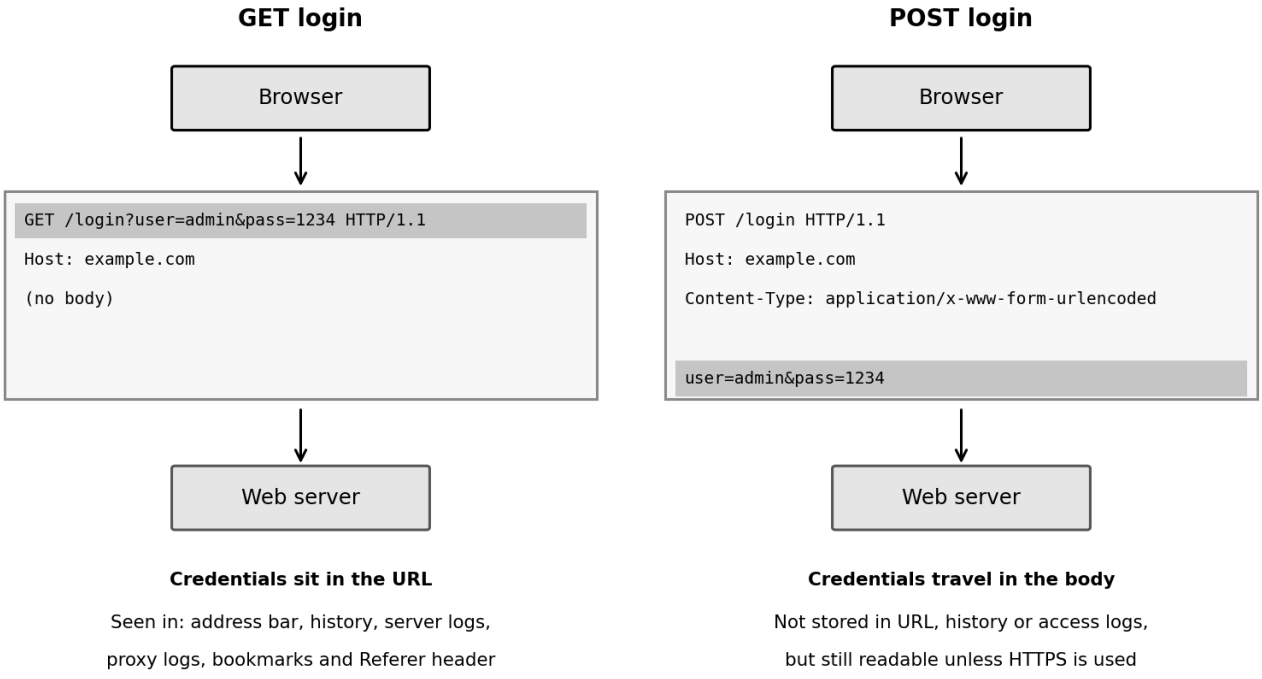


Figure 3: How the login credentials travel with GET and with POST

2. Analyze which method is more suitable for login and why.

POST is the suitable choice. A login request carries a secret, and it also changes state because it creates a session. GET is meant for safe, repeatable requests, so it is a poor match. With POST the password does not appear in the address bar, in the browser history or in normal access logs, and the request will not be re-sent by accident through a bookmark or a prefetch.

3. Identify security risks in using GET for authentication.

- The username and password are visible on screen, so anyone looking over the shoulder can read them.
- They are saved in browser history and bookmarks, and may be auto-completed later.

- Web servers, proxies and load balancers write full URLs into their logs, which spreads the password to many places.
- The URL can leak to other sites through the Referer header when the user clicks an outside link.
- Links can be cached or shared by mistake, and a login can be triggered by a crafted link, which makes CSRF and replay attacks easier.

4. Suggest best practices for secure login implementation.

- Use POST and serve the whole site over HTTPS (TLS). POST alone is only "moderate" security, because the body is readable on plain HTTP.
- Store passwords only as salted hashes made with bcrypt, scrypt or Argon2, never as plain text.
- Add CSRF tokens, set cookies with Secure, HttpOnly and SameSite flags, and send **`Cache-Control: no-store`** on login pages.
- Limit failed attempts with rate limiting or lockouts, and offer multi-factor authentication.
- Validate input and use prepared statements to block SQL injection. Show a generic error such as "invalid username or password".
- Redirect after a successful login (Post/Redirect/Get) so refreshing the page does not re-submit the credentials.

> **Conclusion:** POST over HTTPS is the correct way to build a login. GET exposes credentials in too many places to be acceptable, and POST on its own is not enough without encryption and the other safeguards above.

Question 4: HTML Code Inspection

1. Identify missing closing tags.

Two closing tags are missing: **`</h1>`** after "Welcome" and **`</p>`** after "This is a paragraph". The html, head, title and body tags are closed correctly.

2. Analyze issues with the <img> tag.

- There is no **`alt`** attribute. Screen readers get nothing to read, the text is missing if the image fails to load, and search engines learn nothing about the picture. It is also a WCAG accessibility failure.
- There is no **`width`** and **`height`**. The browser cannot reserve space before the image arrives, so the page jumps when it loads (layout shift).
- The tag itself is fine in HTML5, because img is a void element and does not need a closing tag. In XHTML it would have to be written as **`<img ... />`**.
- The relative path **`image.jpg`** will show a broken-image icon if the file is not in the same folder as the page.

3. Determine how browsers will handle these errors.

The browser shows no error message. The HTML5 parser recovers by its own rules. Since the h1 is never closed, the p tag that follows becomes a child of the h1, and the img goes inside that p. Everything stays open until the parser reaches **`</body>`**, where all open elements are closed together. The visible effect is that the paragraph text inherits the heading style and appears large and bold, as in Figure 5. Nesting a paragraph inside a heading is also invalid HTML, so a validator would report it even though the page still displays.

4. Suggest a corrected version of the code.

The corrected code closes both tags, gives the image a text alternative and fixed dimensions, and adds a language and character set for a complete page.

Given code

```
<!DOCTYPE html>
<html>
<head>
  <title>Sample</title>
</head>
<body>
<h1>Welcome                    (1)
<p>This is a paragraph         (2)
<img src="image.jpg">          (3)
</body>
</html>
```

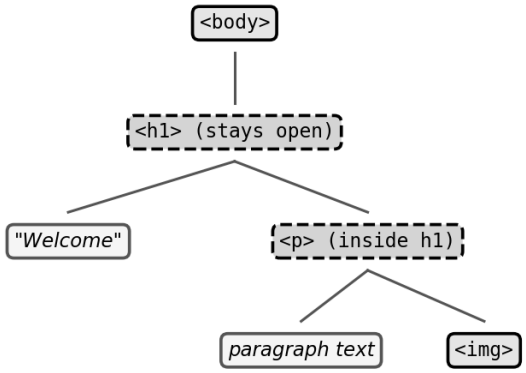
Corrected code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Sample</title>
</head>
<body>
  <h1>Welcome</h1>
  <p>This is a paragraph</p>
  <img src="image.jpg"
       alt="Sample image"
       width="400" height="300">
</body>
</html>
```

1 = <h1> not closed 2 = <p> not closed 3 = <img> without alt, width, height

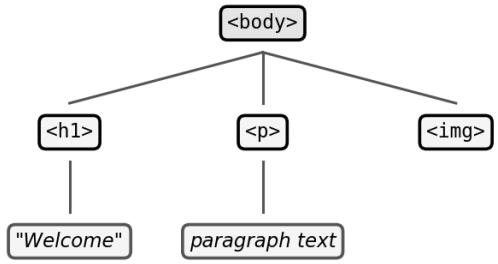
Figure 4: Given HTML code with its errors marked, and the corrected version

Tree the browser builds from the given code



Paragraph and image end up nested inside the heading

Tree after the corrections



Heading, paragraph and image are separate siblings

Figure 5: How the browser builds the tree before and after the corrections

Conclusion: The page "works" only because browsers are forgiving. The missing closing tags quietly nest the paragraph inside the heading and change its look, and the image lacks accessibility and layout information. The corrected code fixes all three problems.

Question 5: Browser-Server Communication Flow

1. Interpret each step in the communication flow.

Step	What actually happens
1. User enters a URL	The browser splits the URL into protocol, host name and path, and checks its own cache and HSTS list before going to the network.
2. DNS lookup	The host name (for example www.example.com) is converted into an IP address, because computers route traffic by IP and not by name.
3. HTTP request is sent	The browser opens a TCP connection to that IP (with a TLS handshake for HTTPS) and sends a request such as GET /index.html along with headers like Host and User-Agent.
4. Server processes request	The web server finds the right resource or application route, runs the code, reads from the database or files and prepares the output.
5. Response is returned	The server sends back a status line, headers and the body, usually HTML. The browser then asks for extra files such as CSS, JavaScript and images.
6. Browser renders page	The browser builds the DOM from HTML and the CSSOM from CSS, runs JavaScript, calculates the layout and finally paints the page on screen.

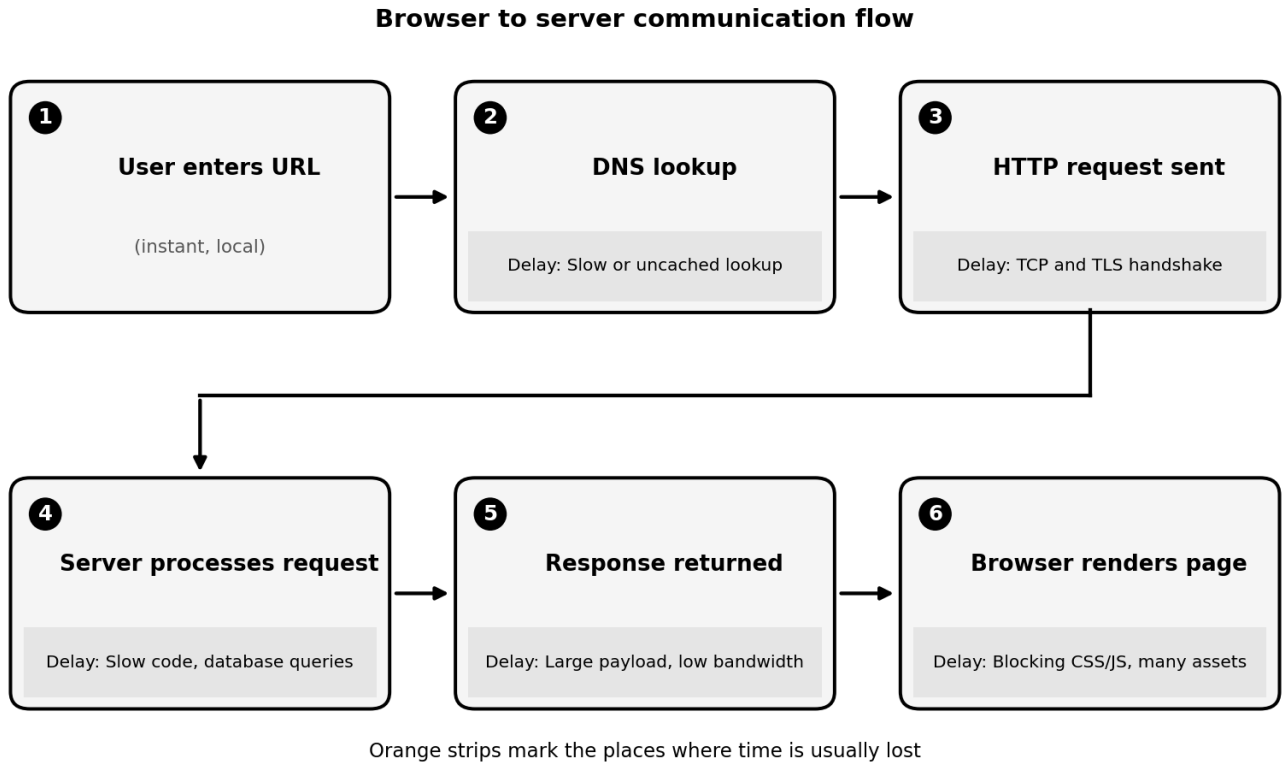


Figure 6: The six steps of the request, with the usual delay points marked

2. Identify where delays can occur.

- **DNS lookup:** an uncached name has to be resolved through several servers, which adds round trips.
- **Connection setup:** the TCP and TLS handshakes need extra round trips before any data moves, and a long distance to the server makes each trip slower.
- **Server processing:** slow code, heavy database queries or an overloaded server delay the first byte.
- **Response transfer:** large HTML, uncompressed files and big images take time on a slow network.
- **Rendering:** scripts and style sheets that block rendering, and a high number of extra requests, hold up the final display.

3. Analyze the role of DNS in the process.

DNS works like the phonebook of the internet. People remember names, but routers deliver packets by IP address, so nothing else can happen until the name is translated. The browser first checks its own cache, then the operating system cache, and then asks the resolver of the ISP. If the resolver does not know the answer, it walks down the hierarchy: root server, then the TLD server (such as .com), then the authoritative server of the domain, which gives the final IP. The result is cached for the time given by its TTL, which is why repeat visits are quicker. A slow or failing DNS server delays or completely blocks the page, even if the web server itself is healthy.

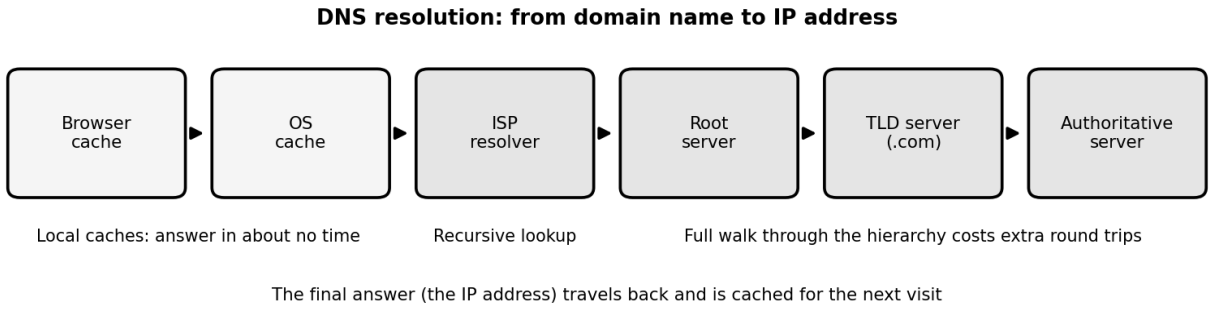


Figure 7: DNS resolution path from local caches to the authoritative server

4. Suggest optimizations to improve performance.

- Speed up DNS with a fast resolver, a sensible TTL, and hints such as **`dns-prefetch`** and **`preconnect`** for the domains the page will use.
- Use a CDN so content is served from a location near the user, and enable HTTP/2 or HTTP/3 with keep-alive connections.
- Cache on the server side (application and database caching, proper indexes) to shorten processing time.
- Compress responses with gzip or Brotli, minify HTML, CSS and JavaScript, and use modern image formats.
- Set cache headers such as Cache-Control and ETag so the browser can reuse files.
- Load non-critical scripts with **`async`** or **`defer`**, lazy-load images, and reduce redirects and the number of requests.

> **Conclusion:** A page load is a chain of six steps, and the page is only as fast as its slowest link. DNS, connection setup, server time, file size and rendering are the main places to look. Caching, a CDN and lighter files improve almost every step.